Análisis Exploratorio, Limpieza, Codificación, Balanceo y Ranking de Atributos sobre el Dataset Home Credit Default Risk

Mondragon Contreras, Sergio
Curso de Machine Learning
Tarea 2 – 31 de julio de 2026

***Abstract*—Resumen pendiente de completar.**

***Index Terms*—Home Credit Default Risk, EDA, calidad de datos, balanceo de clases, selección de atributos, PCA**

I. Introducción

Este informe desarrolla la Tarea 2 del curso de Machine Learning: un análisis exploratorio, de calidad, limpieza, codificación, balanceo y selección de atributos sobre el dataset público de la competencia Kaggle *Home Credit Default Risk*. El desarrollo completo (código, resultados intermedios y figuras) se produjo en un notebook Jupyter (`notebooks/tarea2_home_credit.ipynb`) y se resume en este documento siguiendo la estructura y el formato exigidos por el enunciado de la tarea.

Estado de este informe: las secciones IV-A, IV-B y IV-C (pasos 1 a 3 del procedimiento) están desarrolladas con resultados reales, obtenidos ejecutando el notebook contra el archivo de datos completo. Las secciones IV-D a IV-H (pasos 4 a 8) están en desarrollo activo y se completan en una iteración posterior de este mismo documento.

II. El Problema

Se cuenta con datos reales de solicitudes de crédito de la entidad Home Credit, incluyendo si cada solicitante terminó incumpliendo con su pago (columna `TARGET`). El problema, en términos propios, consiste en explorar a fondo ese conjunto de datos — entender su estructura, medir su calidad, corregir sus defectos, transformarlo a una forma que un algoritmo de aprendizaje pueda consumir, y determinar qué atributos concentran la mayor capacidad para anticipar si un cliente va a incumplir — como paso previo, necesario y no trivial, a la construcción de cualquier modelo predictivo de riesgo crediticio. Le importa a Home Credit porque su modelo de negocio depende de extender crédito responsablemente a personas sin historial crediticio tradicional: un modelo de riesgo deficiente termina rechazando a buenos pagadores o aprobando a quienes no podrán pagar.

III. Metodología de Trabajo

El procedimiento sigue ocho pasos, en este orden, cada uno con su propio ciclo de código, explicación y conclusión parcial, según lo exigido en la sección PROCEDIMIENTO de `hw02-2026-2.docx`:

1) Descripción del problema y origen de los datos.
2) Descripción general del dataset (forma, tipos de dato, rangos).
3) Análisis de la variable `TARGET` (balance de clases).
4) Calidad de los datos (nulos, outliers, duplicados) y primer EDA.
5) Limpieza de datos y segundo EDA (post-limpieza).
6) Conversión de atributos categóricos a numéricos y tercer EDA.
7) Balanceo de clases.
8) Ranking de atributos por capacidad predictiva (técnicas *Filter*), con histogramas por clase, dispersión y PCA.

Los pasos 4 y 5 (Calidad y Limpieza) están explícitamente ponderados con mayor peso en la evaluación de la tarea, por lo que reciben el mayor nivel de detalle. El paso 8 depende de que el paso 7 (balanceo) ya se haya completado.

IV. Desarrollo de la Solución

A. Descripción del problema

El archivo `application_train.csv` proviene de la competencia pública de Kaggle *Home Credit Default Risk* (2018). Home Credit es una institución financiera no bancaria fundada en 1997 en la República Checa, que opera en 14 países y se especializa en otorgar préstamos a personas con historial crediticio limitado o inexistente. Al año 2018 (fecha de referencia de la competencia), el grupo reportaba más de 29 millones de clientes y más de 160 millones de préstamos otorgados, la mayoría en Asia.

Home Credit publicó el dataset en Kaggle retando a la comunidad a mejorar sus modelos de predicción de riesgo de incumplimiento (*default*). El objetivo declarado de la competencia era doble: que clientes capaces de pagar no sean rechazados injustamente por falta de historial crediticio tradicional, y que los préstamos que sí se otorguen vengan con un capital, plazo y calendario de pagos que el cliente pueda sostener realmente.

Cada fila de `application_train.csv` es una solicitud de préstamo, identificada por `SK_ID_CURR`, con datos

estáticos del solicitante al momento de la solicitud (demográficos, financieros, de vivienda, y puntajes normalizados de fuentes externas) y la columna `TARGET`, que vale 1 si el cliente tuvo un pago tardío significativo en alguno de los primeros plazos del crédito, y 0 en cualquier otro caso.

La métrica de evaluación de la competencia original era el área bajo la curva ROC (ROC-AUC) entre la probabilidad predicha de `TARGET=1` y el valor observado — se menciona como contexto porque condiciona qué significa "buen modelo" en este dominio: dado que el dataset está desbalanceado (Sección IV-C), el *accuracy* sería una métrica engañosa.

Nota de alcance: el dataset completo de Kaggle incluye 10 archivos. Esta tarea, por instrucción explícita del enunciado, usa **solo** `application_train.csv`; los demás archivos se mencionan aquí únicamente como contexto de origen, no se cargan en el análisis.

B. Descripción general del dataset

El dataset tiene **307 511 filas y 122 columnas**. La Tabla I resume la composición por tipo de dato.

TABLE I: Tipos de dato en `application_train.csv`

Tipo	N. de columnas
Numérico (`int64`/`float64`)	106
Categórico (`object`/`str`)	16

Dentro de las 106 columnas numéricas hay una mezcla heterogénea: identificadores (`SK_ID_CURR`), indicadores binarios 0/1 (`FLAG_MOBIL`, `FLAG_EMP_PHONE`), conteos enteros (`CNT_CHILDREN`, `CNT_FAM_MEMBERS`), montos continuos (`AMT_INCOME_TOTAL`, `AMT_CREDIT`) y variables de tiempo relativas expresadas en días negativos (`DAYS_BIRTH`, `DAYS_EMPLOYED`). Esta heterogeneidad importa: no se puede tratar toda columna "numérica" con la misma estrategia de limpieza o escalado, aunque `pandas` las tipifique igual.

Entre las 16 categóricas, la cardinalidad es muy dispar: la mayoría tiene entre 2 y 8 categorías (p. ej. `NAME_CONTRACT_TYPE`, `CODE_GENDER`), mientras que `ORGANIZATION_TYPE` tiene 58 categorías distintas. Esto condiciona directamente la Sección IV-F: las de baja cardinalidad son candidatas naturales a codificación One-Hot sin explotar en dimensionalidad, mientras que `ORGANIZATION_TYPE` necesita una técnica como Frequency o Binary Encoding.

Influencia en las secciones posteriores. 307 511 filas es un volumen amplio para entrenar un modelo de clasificación binaria robusto — en principio suficiente para que la validación cruzada y las técnicas de balanceo (SMOTE, Sección IV-G) tengan margen para generar/seleccionar ejemplos sin quedarse sin datos. Sin embargo, el volumen total *no resuelve* el problema de desbalance de clases: el tamaño de muestra grande ayuda a la estabilidad estadística general, pero no sustituye a una estrategia explícita de balanceo.

C. Análisis del Target

La Tabla II y la Fig. 1 resumen la distribución de la variable objetivo.

TABLE II: Distribución de la variable `TARGET`

Clase	Descripción	% del total
0	No incumple (repaga a tiempo)	91.93 %
1	Incumple (pago tardío significativo)	8.07 %

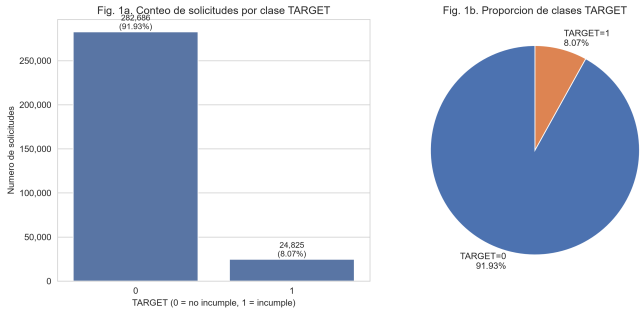


Fig. 1: Conteo y proporción de solicitudes por clase `TARGET`.

La clase `TARGET=0` representa 282 686 solicitudes (91.93 %) y la clase `TARGET=1` representa 24 825 solicitudes (8.07 %) — una razón de aproximadamente 11.4 solicitudes que no incumplen por cada 1 que sí incumple.

¿Están los datos balanceados? No. El criterio operativo de este informe es que una clase minoritaria por debajo del 10–20 % del total ya constituye desbalance; aquí la clase minoritaria está en 8.07 %, por debajo incluso del límite más estricto de ese rango. Es un desbalance real y significativo, no marginal.

Qué es el balanceo de datos y cómo afecta los resultados. El balanceo de datos es el conjunto de técnicas que corrigen, en los datos de entrenamiento o en la función de pérdida del modelo, la disparidad en la cantidad de ejemplos entre clases, de forma que el algoritmo no termine optimizando casi exclusivamente para la clase mayoritaria. El riesgo concreto de no balancear un dataset con esta proporción es que un modelo puede alcanzar una exactitud engañosamente alta ($\sim$92 %) con la estrategia trivial de predecir siempre `TARGET=0`. En términos de métricas de clasificación, el desbalance típicamente hunde el *recall* y el F1-score de la clase minoritaria mientras la precisión global se mantiene artificialmente alta — exactamente el motivo por el que la competencia original evalúa con ROC-AUC y no con *accuracy*.

Postura propia. En un problema de riesgo crediticio como este, el costo de los dos tipos de error no es simétrico: dejar pasar un cliente que va a incumplir (falso negativo) le cuesta dinero real a Home Credit, mientras que rechazar por error a un cliente que sí habría pagado (falso positivo) es un costo de oportunidad, no una pérdida directa. Esto sugiere que el balanceo no debería tratarse solo como un paso técnico para "limpiar" el desbalance estadístico, sino como una decisión alineada con esa asimetría de costos — preferir una técnica que privilegie el *recall* de la clase `TARGET=1` sobre la precisión general. Este criterio se retoma en la Sección IV-G.

D. Calidad de los Datos

[Pendiente de desarrollo — paso 4 del procedimiento. Se documentarán aquí: % de nulos por columna, filas con nulos, outliers/valores inconsistentes, duplicados y columnas de una sola categoría, junto con el primer EDA completo (Anexo 1 de `hw02-2026-2.docx`).]

E. Limpieza de Datos

[Pendiente de desarrollo — paso 5. Incluirá el tratamiento del valor centinela `365243` en `DAYS_EMPLOYED` y el criterio de retención de `EXT_SOURCE_1` pese a su 56 % de nulos.]

F. Conversión de Atributos Categóricos en Numéricos

[Pendiente de desarrollo — paso 6. Aplicará las técnicas del Anexo 2 (Label, One-Hot, Ordinal, Frequency, Binary) según el tipo de cada una de las 16 columnas categóricas.]

G. Balanceo de Datos

[Pendiente de desarrollo — paso 7. Técnica candidata: SMOTE o `class_weight='balanced'` (ver justificación preliminar en Sección IV-C y Anexo 3 de `hw02-2026-2.docx`).]

H. Ranking de Atributos según su Capacidad Predictiva

[Pendiente de desarrollo — paso 8, condicionado a que el paso 7 esté completo. Usará técnicas Filter *(`f_classif`, `mutual_info_classif`, `chi2`, correlación), con histogramas por clase, dispersión de atributos seleccionados, y PCA sobre atributos originales vs. seleccionados.]*

V. Resultados

[Pendiente — se completa cuando cierren los 8 pasos del desarrollo.]

VI. Conclusiones

[Pendiente — con indicadores numéricos, no opiniones. Hasta ahora, verificado: el dataset tiene 307 511×122 (Sección IV-B) y un desbalance de clases de 11.4:1 que requiere corrección explícita antes de modelar (Sección IV-C).]

VII. Recomendaciones

[Pendiente — mínimo 3, con análisis de viabilidad, se redactan al cierre del desarrollo.]

VIII. Código Fuente

```python
class HomeCreditAnalysis:
    """Orquesta el analisis completo de application_train.csv
    (Tarea 2). Cada metodo publico implementa un paso del
    procedimiento exigido por hw02-2026-2.docx. El estado se
    guarda como atributos de instancia para que los pasos
    posteriores lo reutilicen sin recibirlo/devolverlo por
    parametro."""

    def __init__(self, data_path="../data/application_train.csv",
                 figures_dir="../output/figures",
                 tables_dir="../output/tables", dpi=300):
        self.data_path = data_path
        self.figures_dir = figures_dir
        self.tables_dir = tables_dir
        self.dpi = dpi
        self.df = None
        self.resumen_tipos = None
        self.cardinalidad = None
        self.numeric_summary = None
        self.resumen_target = None

    def cargar_datos(self):
        """Paso 1-2 (carga): unico archivo en el alcance."""
        self.df = pd.read_csv(self.data_path)
        print(f"application_train.csv -> filas: "
              f"{self.df.shape[0]:,}  columnas: {self.df.shape[1]}")
        return self.df

    def describir_dataset(self):
        """Paso 2: tipos de dato, cardinalidad, Tabla I."""
        df = self.df
        # pandas 3.x reporta columnas de texto como dtype "str"
        # (antes "object") -- se cuentan ambas etiquetas.
        dtype_counts = df.dtypes.astype(str).value_counts()
        n_numericas = int(dtype_counts.get("int64", 0)
                          + dtype_counts.get("float64", 0))
        n_categoricas = int(dtype_counts.get("object", 0)
                            + dtype_counts.get("str", 0))

        cat_cols = df.select_dtypes(include="object").columns.tolist()
```

```
        self.cardinalidad = (df[cat_cols].nunique()
                             .sort_values(ascending=False))

        self.numeric_summary = (df.select_dtypes(include=[np.number])
                                .describe().T)
        self.numeric_summary["rango"] = (self.numeric_summary["max"]
                                         - self.numeric_summary["min"])

        self.resumen_tipos = pd.DataFrame({
            "tipo": ["numerico (int64/float64)",
                     "categorico (object/str)"],
            "n_columnas": [n_numericas, n_categoricas],
        })
        self.resumen_tipos.to_csv(
            f"{self.tables_dir}/tabla01_tipos_de_dato.csv", index=False)
        return self.resumen_tipos, self.cardinalidad

    def analizar_target(self):
        """Paso 3: balance de clases, Tabla II y Fig. 1."""
        df = self.df
        target_counts = df["TARGET"].value_counts().sort_index()
        target_pct = (target_counts / target_counts.sum()
                      * 100).round(2)
        self.resumen_target = pd.DataFrame({
            "clase": target_counts.index,
            "conteo": target_counts.values,
            "porcentaje": target_pct.values,
        })
        self.resumen_target.to_csv(
            f"{self.tables_dir}/tabla02_balance_target.csv",
            index=False)

        fig, axes = plt.subplots(1, 2, figsize=(11, 5))
        sns.barplot(x=self.resumen_target["clase"].astype(str),
                    y=self.resumen_target["conteo"], ax=axes[0],
                    color="#4C72B0")
        axes[0].set_title(
            "Fig. 1a. Conteo de solicitudes por clase TARGET")

        etiquetas = [f"TARGET={c}\n{p}%" for c, p in
                     zip(self.resumen_target["clase"],
                         self.resumen_target["porcentaje"])]
        axes[1].pie(self.resumen_target["conteo"], labels=etiquetas,
                    colors=["#4C72B0", "#DD8452"], startangle=90)
        axes[1].set_title("Fig. 1b. Proporcion de clases TARGET")

        plt.tight_layout()
        plt.savefig(f"{self.figures_dir}/fig01_target_balance.png",
                    dpi=self.dpi, bbox_inches="tight")
        plt.close(fig)
        return self.resumen_target

    # TODO paso 4: calidad_de_los_datos(self)
    # TODO paso 5: limpiar_datos(self)
    # TODO paso 6: convertir_categoricas(self)
    # TODO paso 7: balancear_clases(self)
    # TODO paso 8: ranking_atributos(self)
```

Listing 1: Clase `HomeCreditAnalysis` – un método por paso del procedimiento

```
def mainProcess():
    """
    Punto de entrada unico del analisis (Tarea 2).
    Pasos 1-3 implementados y verificados contra el dataset
    completo (307,511 x 122). Pasos 4-8 (calidad, limpieza,
    encoding, balanceo, ranking+PCA) en desarrollo -- ver
    hw02-2026-2.docx, seccion PROCEDIMIENTO.
    """
    analysis = HomeCreditAnalysis()
    analysis.cargar_datos()
    analysis.describir_dataset()
    analysis.analizar_target()

    # TODO paso 4: analysis.calidad_de_los_datos()
```

```
    # TODO paso 5: analysis.limpiar_datos()
    # TODO paso 6: analysis.convertir_categoricas()
    # TODO paso 7: analysis.balancear_clases()
    # TODO paso 8: analysis.ranking_atributos()

    return analysis


if __name__ == "__main__":
    mainProcess()
```

Listing 2: mainProcess – orquestador principal del análisis

REFERENCES

[1] M. Kuhn and K. Johnson, *Feature Engineering and Selection: A Practical Approach for Predictive Models*. Boca Raton, FL, USA: Chapman & Hall/CRC Press, 2019.

[2] H. He and Y. Ma, *Imbalanced Learning: Foundations, Algorithms and Applications*. Hoboken, NJ, USA: Wiley-IEEE Press, 2013.